Programmier-sprachen 3

Computational Thinking 3

Benjamin Thomsen

Computational Thinking 3

„Computational Thinking („informatisches Denken“) ist eine strukturierte Methode, um komplexe Probleme so zu analysieren und zu lösen, dass die Lösung im Idealfall auch von einem Computer umgesetzt werden kann. Es funktioniert jedoch völlig ohne Computer als reines Werkzeug im Kopf.

Der Ansatz verbindet sich eng mit dem Coding (Programmieren), setzt echtes Programmieren aber nicht zwingend voraus – es bildet vielmehr die logische Grundlage dafür“.

Benjamin Thomsen

Ziel des Kurses

- Technisches Vokabular aufbauen
- Technologien verstehen, nicht nur benutzen
- KI richtig verwenden

Überblick

- Arbeiten im Terminal ohne KI und mit GitHub
  - Funktionen, Parameter, etc.
- Einführung in die Arbeitsweise mit KI
  - Allgemeines zu Coding mit KI
  - Wie kommuniziere ich mit der KI
  - Ergebnis kritisch hinterfragen
- UIs ohne Zustand (Browser)
  - Ereignisse, Listener, Module
- Zustände, States (Framework)
  - Frontend-Frameworks
- Persistenz
  - Daten speichern
  - Speichermöglichkeiten im Browser
- Fremde Daten per API
  - fetch, async
  - Lade- und Fehlerzustände

Semesteraufgabe

- Eine Weltkarte mit euren eigenen Orten
- Aus Breiten- und Längengrad wird ein Punkt auf der Karte
- Das Projekt wächst von Woche zu Woche
  - Orte anzeigen, Orte anlegen, speichern, Daten aus dem Netz
- Eigene Sammlung an Orten + spezifische Informationen

Grober Ablauf

- Ergebnisse der Vorwoche zeigen
- Theorievortrag
- Arbeitsweise mit KI
- Werkstatt / praktische Übung
- Wochenaufgabe

Bewertung

„Ihr sollt verstehen was ihr macht bzw.
verstehen was eine KI für euch macht.“

Abgabe jede Woche

- Code als GIT-Commit
- README.md: Was es tut, wie man es startet, immer aktuell
- ENTSCHEIDUNGEN.md: Beantwortung der Fragen aus der Hausaufgabe

Abgabe am Ende

- Das Projekt als Repository
- Präsentation des Projekts

Review-Paare

- Feste Paare fürs ganze Semester
- Ab T2: die Abgabe des Partners lesen, einen konkreten Satz dazu schreiben
- In der Werkstatt: gegenseitig Helfen
- Bei uns: drei Paare und eine Dreiergruppe

Theorie-Input

Was ist Code?

Quellcode → Interpreter / Compiler → Maschinencode

- Quellcode: der Text, den ihr schreibt
- Maschinencode: das, was der Rechner ausführt
- Compiler: übersetzt vorher, alles auf einmal
- Interpreter: liest und führt direkt aus
- Transpiler: übersetzt in anderen Quellcode

Wo kann Code ausgeführt werden?

→ Laufzeitumgebung

- Terminal (Node.js)
- Browser

Demo

Syntax vs. Semantik

Syntaxfehler — das Programm läuft gar nicht erst

```
const x = ;
// SyntaxError: Unexpected token ,;
```

Semantikfehler — das Programm läuft,
macht aber etwas anderes als gemeint

```
function mittelwert(a, b) {
  return (a + b) / 2;
}
mittelwert(3, 4);         // 3.5   richtig
mittelwert("3", "4");     // 17    läuft. ist falsch.
```

Demo

vs. Laufzeitfehler

Code läuft bis zum Fehler

```
console.log("hallo")

function mittelwert(a, b) {
  return (a + b) / 2;
}

mittelwerte(3, 4); // 3.5   richtig
```

Demo

Wichtige Begriffe

- Variable und Zuweisung
- Benennung und Schreibweise
- Anweisung und Ausdruck
- Block und Gültigkeit
- Kommentare

Wichtige Begriffe

Variablen und Zuweisung

```
// Deklaration und Zuweisung mit let
let stadt = 'Stuttgart';

// Wert nachträglich ändern
stadt = 'Berlin';

// Konstante anlegen (kann nicht geändert werden)
const pi = 3.14159;

// Ohne Wert ist sie undefined
let stadt;
```

Wichtige Begriffe

Benennung

```
// Variablen sollen beschreiben
let userAge = 25;

// Funktionen mit einem Verb beginnen
fetchData();
updateUser();

// Abkürzungen lieber vermeiden
const imageWidth;
const imgW;
```

Wichtige Begriffe

Schreibweise

```
// camelCase: Standard für Variablen, Funktionen,
// Methoden, Objekt-Eigenschaften und Parameter.
let userName;
function fetchData() {}

// PascalCase: Klassen, Konstruktor-Funktionen,
// Framework-Komponenten
class UserProfile {}
function Button() {}

// UPPER_SNAKE_CASE: Unveränderliche globale
// Konstanten und feste Konfigurationswerte.
const MAX_RETRIES = 3;
const API_URL = "https://api.com/*";

// kebab-case: Dateinamen, CSS-Klassen,
// HTML-Attribute und npm-Pakete
// (als JS-Variable syntaxtechnisch nicht erlaubt).
user-profile.js
npm install my-package
```

Wichtige Begriffe

Anweisung (Statement) und Ausdruck (Expression)

Eine Anweisung tut etwas

```
const x = 5 + 3; (eine Variablendeklaration / Zuweisung)
console.log("Hallo"); (eine Ausgabe)
if (x > 5) { ... } (ein Bedingung)
```

Ein Ausdruck ist ein Codeteil

```
5 + 3 (ergibt die Zahl 8)
"Hallo " + "Welt" (ergibt den Text "Hallo Welt")
x > 5 (ergibt wahr oder falsch)
```

Wichtige Begriffe

Block und Gültigkeit

```
{ // Start von Block 1 (Äußerer Block)
  let a = 10;

  { // Start von Block 2 (Innerer Block)
      let b = 20;
      console.log(a); // Gültig! Gibt 10 aus (Zugriff auf äußeren Block).
      console.log(b); // Gültig! Gibt 20 aus.
  } // Ende von Block 2 -> Variable b wird hier gelöscht!

  console.log(a); // Gültig! Gibt 10 aus.
  console.log(b); // FEHLER! b ist hier nicht mehr gültig/bekannt.
} // Ende von Block 1 -> Variable a wird hier gelöscht!
```

Wichtige Begriffe

Kommentare

```
// Wir nutzen hier eine klassische for-Schleife
// statt .forEach(), da die Performance bei Arrays
// mit über 1 Mio. Einträgen kritisch ist.
for (let i = 0; i < riesigesArray.length; i++) {
  // ...
}
```

```
// Timeout von 300ms ist nötig, weil die API
// weil uns die API sonst blockiert.
setTimeout(() => {
  fetchUserData();
}, 300);
```

Determinismus

→ Eigenschaft von eurem Code-Snippet?

Determinismus

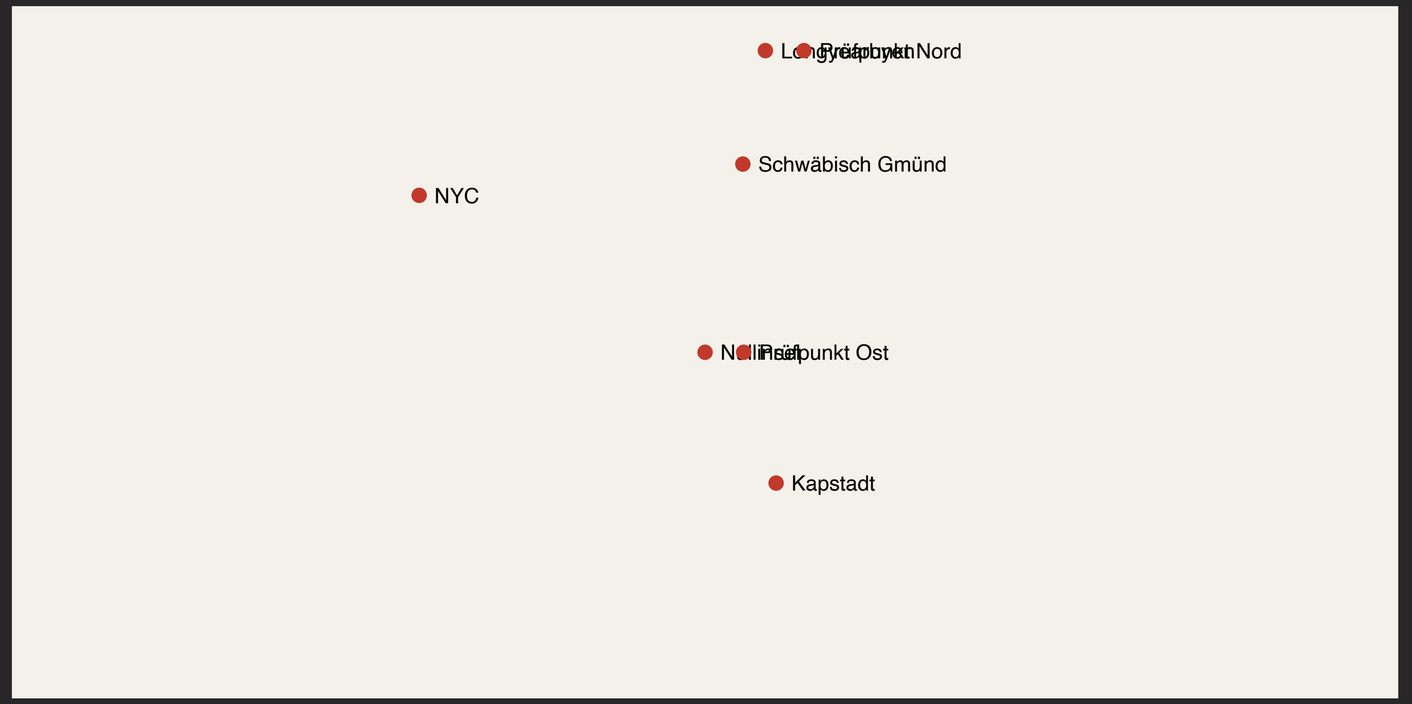
→ Eigenschaft von eurem Code-Snippet?

- Vorhersehbarkeit
- Testbarkeit
- Nicht-deterministisch
    - Zufallszahlen
    - Datum und Uhrzeit
    - Asynchronität und APIs
    - (Nutzung von KI)

10 Minuten Pause

Übung

Wie kann man einen Längen- und Breitengrad in eine Pixel-Position im Bild umrechnen.



Spezifikation der Aufgabe

- Beschreiben was gegeben ist
- Beschreiben was die Aufgabe macht
- Beschreiben was dabei rauskommt

Kein Code – Spezifikation vor Code!

Vergleich

- Vergleich im Review-Paar
- Ähnliches Ergebnis?
- Welche Unterschiede?

Beispiel Spezifikation

Umrechnung von RGB-Farbwerten in einen Hex-Farbcode

- Rot, Grün und Blau von 0 bis 255
- Hex-Format mit 6 / 7 Zeichen: #RRGGBB
- 0 = 00 und 255 = FF (Hexadezimal)
- Werte unter 16 erhalten eine führende 0 (z. B. 9 → 09)
- Eine Funktion
- Rot + Grün + Blau → Hex-Farbcode als Text

Mögliche Spezifikation

Mögliche Spezifikation

- Breitengrade von +90° bis -90°
- Längengrade von -180° bis +180°

Mögliche Spezifikation

- Breitengrade von +90° bis -90°
- Längengrade von -180° bis +180°
- Breite und Höhe der Karte 720 × 360 px

Mögliche Spezifikation

- Breitengrade von +90° bis -90°
- Längengrade von -180° bis +180°
- Breite und Höhe der Karte 720 × 360 px
- Links: 0px = -180° und rechts: 720px = +180°
- Oben: 0px = +90° und unten: 360px = -90°

Mögliche Spezifikation

- Breitengrade von +90° bis -90°
- Längengrade von -180° bis +180°
- Breite und Höhe der Karte 720 × 360 px
- Links: 0px = -180° und rechts: 720px = +180°
- Oben: 0px = +90° und unten: 360px = -90°
- Zwei Funktionen, Eingabe / Ausgabe
  - Längengrad + Breite → x in Pixel
  - Breitengrad + Höhe → y in Pixel

Werkstatt

- Funktionen auf Papier
- Arbeiten im Terminal
- Funktionen im Code
- Commit auf Git

Werkstatt

- SVG-Größe (720 x 360px)
- lonToX(-180) → 0px
- lonToX(180) → 720px
- latToY(90) → 0px
- latToY(-90) → 360px



Wochenaufgabe

1 - Eine dritte Funktion, die mit den Orten (Array) rechnet, mit eigenem Commit.

2 - Eigene Orte und die Karte generieren (mindestens 5 Orte)

3 — Zwei Dateien: README.md und ENTSCHEIDUNGEN.md

Wochenaufgabe

README.md

- Wie startet man das Script?

ENTSCHEIDUNGEN.md

1. Was tut dein Programm?
2. Welche Stelle hättest du auch anders lösen können? Wie?
3. Warum hast du es so gelöst?
4. Was fiel dir schwer bei der Aufgabe?
5. Was ist dir unklar geblieben?